Database

What is Database:-

- A **database** is an organized, electronic collection of data that is systematically stored and managed in a computer system.
- It's essentially a repository that allows data to be easily stored, accessed, managed, modified, and updated. Think of it as a highly sophisticated digital filing cabinet where the data is organized in a way that makes it efficient to search, retrieve, and analyze.
- Databases are crucial for almost every modern application, from tracking inventory and customer orders in a business to managing user information for social media sites and banking systems.

A high-performing database is vital for any organization, supporting operations, customer interactions and systems like digital libraries, reservations, and inventory management. Databases are essential because they:

- **Scale efficiently** to handle massive volumes of data.
- **Ensure data integrity** through built-in rules and constraints.
- **Protect data** with secure access controls and compliance support.
- **Enable analytics** by identifying trends and guiding informed business decisions.

1

Manual Management

3

Slow Data Access

Before **Database**

Data Overload

2

Limited Storage

4

1. Manual Management

Before databases, data was stored **manually** on **paper files, registers, or record books**.People had to **write, organize, and search** for data by hand.
Example: In a school, all student details were written in notebooks or ledgers.
Problem: Time-consuming, errors could happen easily, and finding information took a lot of effort.

2. Data Overload

As organizations grew, the **amount of data increased rapidly**.Manual systems could not **handle or organize** such a huge volume of data properly.
Example: A hospital with thousands of patient files faced difficulty managing and updating all records.
Problem: Data became unmanageable, leading to confusion and loss of important information.

3. Slow Data Access

Finding any specific information (like a customer's record) took a **lot of time** because workers had to **search files one by one**.There were **no search tools** like in computers.
Example: If a company wanted to find sales data for a customer, they had to open and check many physical files.
Problem: Decision-making was delayed because data retrieval was too slow.]

4. Limited Storage

Paper files and registers required **physical space** — cupboards, shelves, roms, etc. As data grew, **storage space ran out**.
Example: A bank storing thousands of paper records needed more rooms or even warehouses.
Problem: Difficult to maintain, expensive, and prone to **damage** (fire, insects, or moisture).

- Efficient management
- Prevents overload
- Quick retrieval
- Scalable storage

with Databases

1. Efficient Management

Databases make it easy to **store, organize, and manage** large amounts of data systematically.You can **add, update, delete, or search** for data quickly using software tools.
Example: A school can easily manage all students' records (marks, attendance, fees) in one place.
Result: Saves time and reduces errors.

2. Prevents Overload

Databases can handle **large volumes of data** efficiently without becoming messy or slow. Unlike manual systems, they are designed to **process and organize** huge datasets.
Example: A hospital can safely store years of patient data without any confusion.
Result: Data stays organized even when the amount of information increases.

3. Quick Retrieval

Databases allow **fast searching and fetching** of any required information.Using queries, you can get results in **seconds** instead of hours.
Example: You can instantly find a customer's purchase history using their ID.
Result: Saves time and supports faster decision-making.

4. Scalable Storage

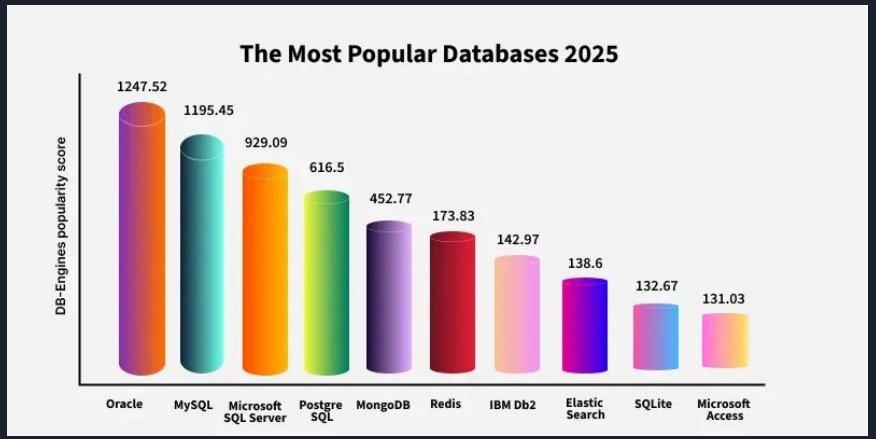
Databases can easily **expand storage capacity** as data grows. No need for extra physical space like in paper files — just **increase digital storage**.
Example: A company can add new products, employees, and sales data without storage problems.
Result: Flexible and future-ready for growing data needs.

Working of Databases:-

Databases work by organizing and storing information in a structured or unstructured format, allowing easy access, retrieval, and modification. At the core of every database system is the **Database Management System (DBMS)**—a software layer that acts as an intermediary between users and the raw data.

- The DBMS handles tasks like querying, updating, deleting and managing access permissions, without requiring users to know the physical details of where data is stored.
- When a user submits a request (such as a search or update), the DBMS processes the query, locates the relevant data, and returns results in a structured format.
- DBMSs provide features like backup, recovery, performance optimization and data security to ensure the system runs efficiently and reliably.

According to industry rankings, the most popular databases of 2025 are:



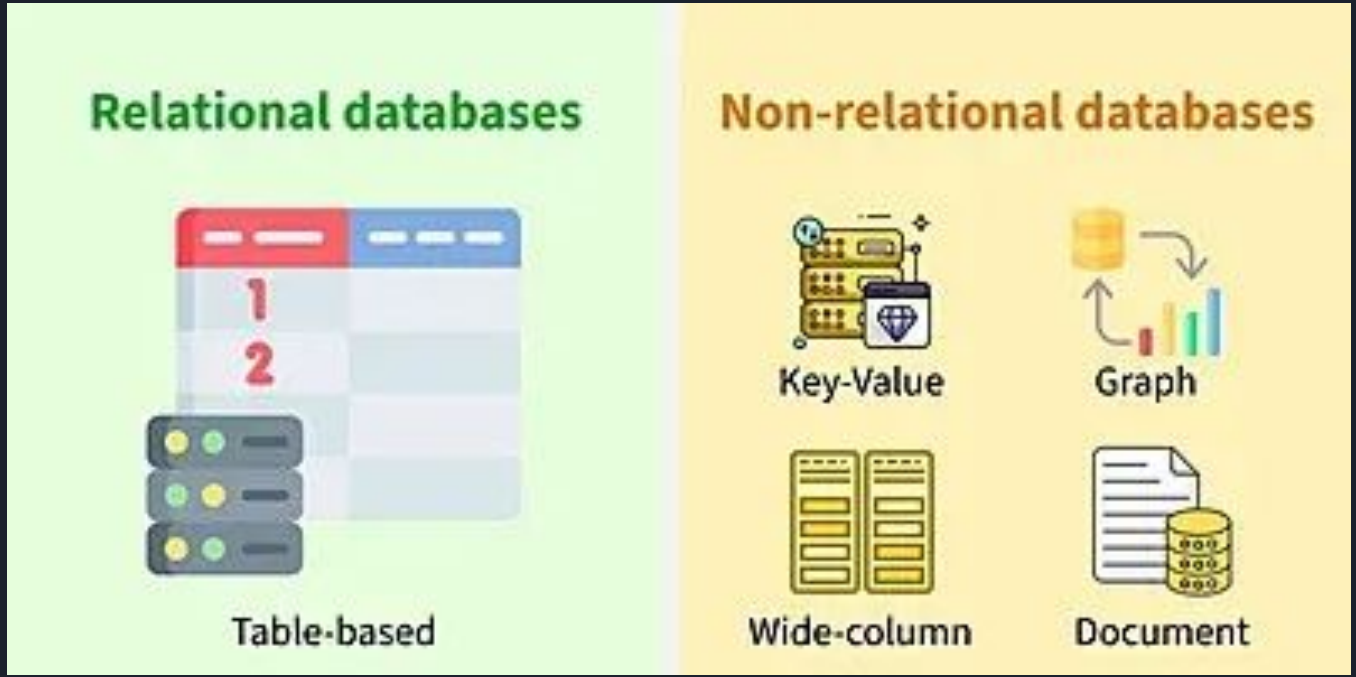
Components of a Database:-

Databases consist of several critical components that work together to **store**, **organize** and **retrieve data** effectively. Here is a detailed explanation of each component:

- **Data**: The actual information stored in the database, such as text, numbers, images, or files.
- **Schema**: The structural blueprint that defines how data is organized—tables, fields, data types, and relationships.
- **DBMS**: The software that manages database operations like storage, retrieval, and security (e.g., MySQL, Oracle).
- **Queries**: Instructions (usually SQL) used to retrieve or manipulate data within the database.
- **Users**: People or systems that interact with the database, each with specific roles and access permissions.

Types of Databases:-

Databases can be classified into two primary types **Relational (SQL)** and **NoSQL Databases**. NoSQL is then further divided into four types: Document-oriented, Key-Value, Wide-Column, and Graph databases.



1. Relational Databases (RDBMS):-

Relational databases organize data into tables made up of rows (records) and columns (fields). They use schemas (blueprints) to define how data is structured and how different tables relate to each other.

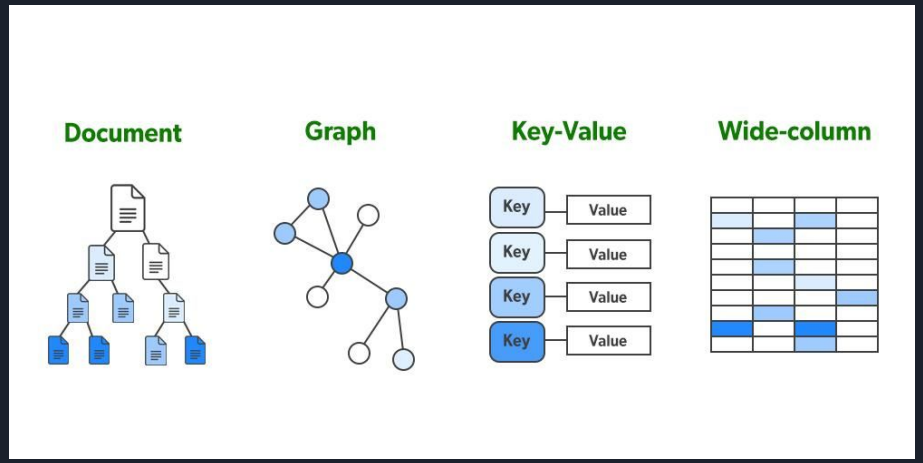
- Strict schema-based structure.
- Primary Keys (unique IDs) and Foreign Keys (relationships between tables).
- Strong ACID compliance (Atomicity, Consistency, Isolation, Durability).
- Ideal for structured data.

2. Non-relational database(NoSQL Databases):-

"NoSQL" stands for "Not Only SQL". These databases are designed to handle unstructured or semi-structured data, such as text, images, videos or sensor data. They don’t rely on the traditional table format.

- Flexible data models (no fixed schema).
- Scales horizontally for high-volume data.
- Often optimized for specific use cases like graphs or time-series data.

Main Sub-Types of NoSQL Databases:



Real-World Applications of Databases:-

Databases are essential part of our life. There are several everyday activities that involve our interaction with databases.

- **Banking**: Stores transactions and account details.
- **Transportation**: Manages bookings and schedules.
- **Education**: Tracks student records and grades.
- **Retail**: Handles inventory and customer orders.
- **Social Media**: Stores user data, messages, and media.
- **Multimedia**: Manages images, audio, and video.
- **Business & Data Science**: Analyzes trends and supports predictions.

Importance of Databases for Different Technology:-

Databases are the engine behind every digital experience—whether you are building an app, training AI models, or running infrastructure at scale. Here is a breakdown of the most suitable database types for various technology domains:

1. Databases for Web Development

Web applications rely heavily on databases to store and manage user data, content, and transactions. Whether it is a blog or a large e-commerce platform, developers typically use:

- **Popular Databases**: MySQL, PostgreSQL, MongoDB, Firebase
- **Use Case**: Dynamic content, authentication, product catalogs

2. Databases for Mobile Development

Mobile apps require fast, lightweight databases optimized for limited device resources and offline access.

- **Popular Databases**: SQLite, Realm, Firebase Realtime DB
- **Use Case**: Local storage, syncing user preferences, offline-first apps

3. Databases for DevOps

DevOps teams manage CI/CD pipelines and infrastructure, where databases must support automation, monitoring, and scale.

- **Popular Databases**: PostgreSQL, Redis, InfluxDB, Cassandra
- **Use Case**: Logging, monitoring metrics, configuration storage

4. Databases for Data Engineering

Data engineers handle massive volumes and real-time pipelines. Their databases must be highly scalable and performant.

- **Popular Databases**: Apache Hadoop (HDFS), Apache Cassandra, Amazon Redshift, Google BigQuery
- **Use Case**: ETL processes, big data pipelines, real-time data streaming

5. Databases for Data Science

Data scientists need flexible querying, data aggregation, and easy integration with tools like Python and R.

- **Popular Databases**: PostgreSQL, MongoDB, Apache Hive, Snowflake
- **Use Case**: Feature extraction, exploratory analysis, modeling datasets

6. Databases for Artificial Intelligence

AI systems depend on structured, unstructured, and streaming data for model training and predictions.

- **Popular Databases**: MongoDB, Apache Cassandra, Google BigQuery, AWS S3 (data lake)
- **Use Case**: Storing training datasets, real-time inference, model feedback loops

7. Database for Cloud Computing

In cloud-native environments, databases must support autoscaling, high availability, and service integration.

- **Popular Databases**: Amazon Aurora, Google Cloud Spanner, Microsoft Azure Cosmos DB
- **Use Case**: SaaS platforms, distributed apps, serverless environments

8. Database for Blockchain/Web3.0

Blockchain-powered systems need tamper-proof, decentralized databases for trustless transactions and transparency.

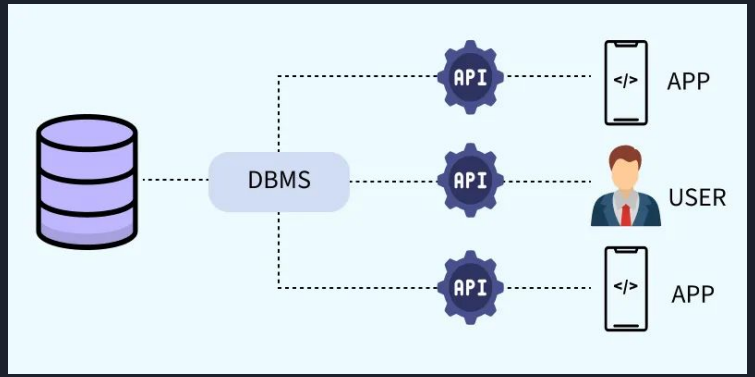
- **Popular Databases**: BigchainDB, IPFS, Ethereum (as data ledger), Chainlink
- **Use Case**: Immutable ledgers, decentralized identity, smart contract data.

DBMS (Database Management System)

Database Management System (DBMS):-

DBMS is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups.



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems:-

Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Types of DBMS:-

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. Relational Database Management System (RDBMS):-

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL oracle, Microsoft SQL Server and Postgresql.

2. NoSQL DBMS:-

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.
- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Advantages of DBMS:-

1) **Data Redundancy Control:**
   a) Same data is not repeated again and again.
   b) Example: Student information stored once and used by all departments.
2) **Data Consistency:**
   a) Because data is stored only once, it remains the same for all users.
   b) No mismatched or duplicate information.
3) **Data Sharing:**
   a) Multiple users can access the same database at the same time.
   b) Example: Teachers and admin both can view student records together.
4) **Data Security:**
   a) Access is given only to authorized users.
   b) Passwords and permissions protect the data.
5) **Backup and Recovery:**
   a) DBMS provides automatic backup and recovery in case of system failure.
6) **Data Integrity:**
   a) Ensures accuracy and correctness of data using rules and constraints.
7) **Easy Data Access:**
   a) You can easily insert, update, delete, or search data using queries (SQL).
8) **Data Independence:**
   a) Changes in data structure do not affect application programs.

Disadvantages of DBMS:-

1. **High Cost:**
   a. DBMS software and hardware are expensive.
   b. Also requires trained staff.
2. **Complexity:**
   a. It is complicated to design and manage compared to manual systems.
3. **Large Size:**
   a. DBMS needs large memory and storage space to run.
4. **Performance Issues:**
   a. For small data or simple tasks, DBMS can be slower than file systems.
5. **Risk of Data Loss:**
   a. If database is corrupted or hacked, large amounts of data can be lost.
6. **Regular Maintenance:**
   a. Needs continuous monitoring, updating, and backup.

SQL

What is SQL:-

- SQL stands for Structured Query Language. It is used for storing and managing data in Relational Database Management System (RDBMS).
- It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- SQL allows users to query the database in a number of ways, using English-like statements.

What are the SQL?

SQL follows the following rules:

- Structure query language is not case sensitive. Generally, keywords of SQL are written in uppercase.
- Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
- Using the SQL statements, you can perform most of the actions in a database.
- SQL depends on tuple relational calculus and relational algebra.

What Can SQL do?

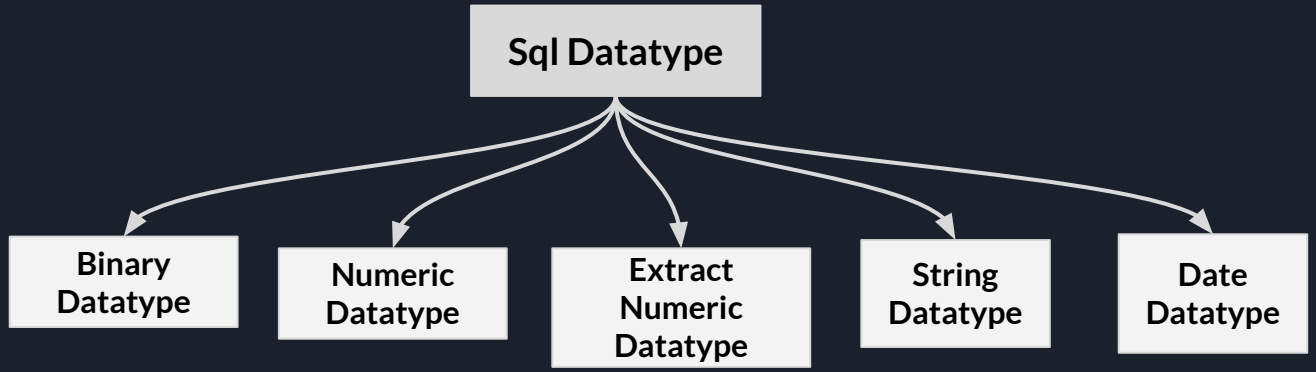
- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

What is Advantages of SQL?

- High speed
- No coding needed
- Well defined standards
- Portability
- Interactive language
- Multiple data view

What is SQL Datatype?

- SQL Datatype is used to define the values that a column can contain.
- Every column is required to have a name and datatype in the database table.

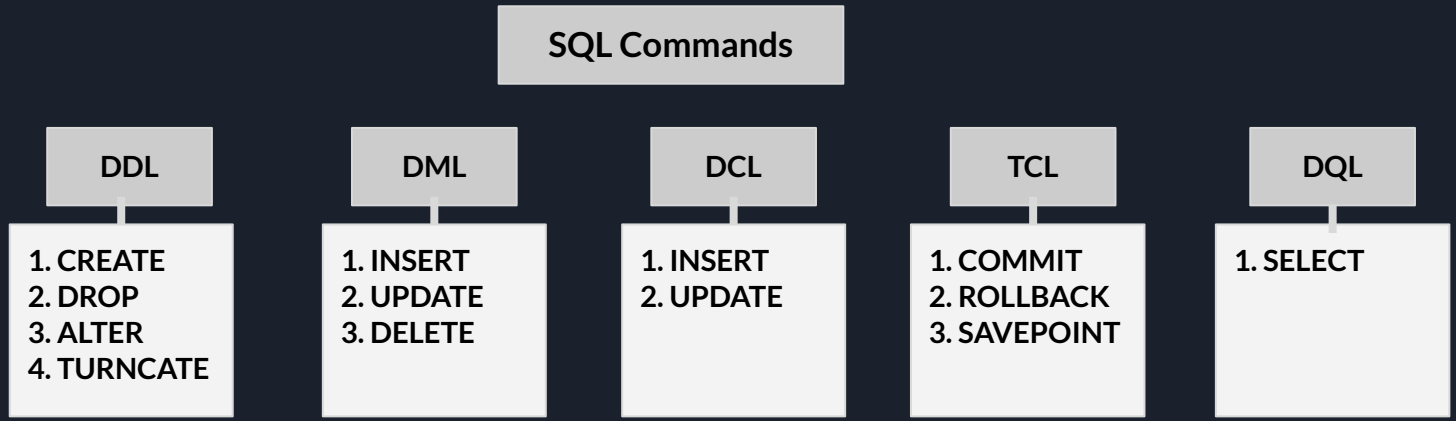


SQL Commands :-

- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.

Types of SQL Commands :-

There are five types of SQL commands: DDL, DML, DCL, TCL, and DQL.



The **DDL** SQL DATABASE Statements:-

1) The **CREATE DATABASE** statement is used to create a new SQL database.

Syntax:
```
CREATE DATABASE databasename;
```
Example:
```
CREATE DATABASE test;
```

Tip: Make sure you have admin privilege before creating any database. Once a database is created, you can check it in the list of databases with the following SQL command: **SHOW DATABASES;**

2) The **USE DATABASE** statement is used to Use a database from created Databases.

Syntax:
```
USE databasename;
```
Example:
```
USE test;
```

3) The **DROP DATABASE** statement is used to Drop an Existing SQL database.

Syntax:
```
DROP DATABASE databasename;
```
Example:
```
DROP DATABASE test;
```

Note: Be careful before dropping a database. Deleting a database will result in loss of complete information stored in the database!

The SQL Table Statements:-

1) The **CREATE TABLE** statement is used to create a new table in a database.

Syntax:

```
CREATE TABLE table_name (column1 datatype,column2 datatype, ....);
```

The column parameters specify the names of the columns of the table.
The datatype parameter specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

Example:

```
CREATE TABLE Persons (PersonID int,LastName varchar(255),FirstName
varchar(255),Address varchar(255),City varchar(255));
```

- The PersonID column is of type int and will hold an integer.
- The LastName, FirstName, Address, and City columns are of type varchar and will hold characters, and the maximum length for these fields is 255 characters.
- The empty "Persons" table will now look like this:

PersonID	LastName	FirstName	Address	City

DATA TYPES IN SQL

The data type of a column defines what value the column can hold: integer, character, money, date and time, binary, and so on.

SQL Data Types:-

Each column in a database table is required to have a name and a data type.
An SQL developer must decide what type of data that will be stored inside each column when creating a table. The data type is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

String Data Types:-

Data type	Description
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum string length in characters - can be from 0 to 65535
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them

Numeric Data Types:-

Data Type	Description
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
INT(*size*)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The *size* parameter specifies the maximum display width (which is 255)
INTEGER(*size*)	Equal to INT(size)
FLOAT(*size*, *d*)	A floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(*p*)	A floating point number. MySQL uses the *p* value to determine whether to use FLOAT or DOUBLE for the resulting data type. If *p* is from 0 to 24, the data type becomes FLOAT(). If *p* is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(*size*, *d*)	A normal-size floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter

Date and Time Data Types:-

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

Create Table Using Another Table:-

- A copy of an existing table can also be created using `CREATE TABLE`.
- The new table gets the same column definitions. All columns or specific columns can be selected.
- If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Syntax:

```
CREATE TABLE new_table_name AS SELECT column1, column2,... FROM
existing_table_name WHERE ....;
```

The following SQL creates a new table called "TestTable" (which is a copy of the "Customers" table):

Example:

```
CREATE TABLE TestTable AS SELECT FirstName, LastName FROM Persons;
```

LastName	FirstName

2) The **DROP TABLE** statement is used to drop an existing table in a database.

Syntax:
```
DROP TABLE table_name;
```
Example:
```
DROP DATABASE Persons;
```

Note: Be careful before dropping a table. Deleting a table will result in loss of complete information stored in the table!

3) The **TRUNCATE TABLE** statement is used to delete the data inside a table, but not the table itself.

Syntax:
```
TRUNCATE TABLE table_name;
```
Example:
```
TRUNCATE DATABASE Persons;
```

4) The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table. The **ALTER TABLE** statement is also used to add and drop various constraints on an existing table.

- ALTER TABLE - ADD Column:-

  To add a column in a table, use the following syntax:

Syntax:
```
ALTER TABLE table_name ADD column_name datatype;
```
Example:
```
ALTER TABLE Persons ADD Email varchar(255);
```

- ALTER TABLE - DROP Column:-

  To delete a column in a table, use the following syntax:

Syntax:

```
ALTER TABLE table_name DROP column_name;
```

Example:

```
ALTER TABLE Persons DROP Email;
```

- ALTER TABLE - RENAME Column:-

  To rename a column in a table, use the following syntax:

Syntax:

```
ALTER TABLE table_name RENAME COLUMN old_name to new_name;
```

Example:

```
ALTER TABLE Persons RENAME COLUMN Email to p_email;
```

- ALTER TABLE - MODIFY Datatype:-

  To modify a column datatype in a table, use the following syntax:

Syntax:

```
ALTER TABLE table_name MODIFY COLUMN column_name datatype;
```

Example:

```
ALTER TABLE Persons MODIFY COLUMN Email varchar(100);
```

SQL Create Constraints:-

Constraints can be specified when the table is created with the `CREATE TABLE` statement, or after the table is created with the `ALTER TABLE` statement.

Syntax:

```
CREATE TABLE table_name ( column1 datatype constraint,column2 datatype
constraint,column3 datatype constraint,....);
```

SQL Constraints:-

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.
The following constraints are commonly used in SQL:

- `NOT NULL` - Ensures that a column cannot have a NULL value
- `UNIQUE` - Ensures that all values in a column are different
- `PRIMARY KEY` - A combination of a `NOT NULL` and `UNIQUE`. Uniquely identifies each row in a table
- `FOREIGN KEY` - Prevents actions that would destroy links between tables
- `CHECK` - Ensures that the values in a column satisfies a specific condition
- `DEFAULT` - Sets a default value for a column if no value is specified
- `Auto Increment` - Auto increment is used to create unique IDs automatically for each new row in a database table.
- `DATE` - DATE is used to store and handle calendar dates.

1. SQL NOT NULL Constraint:-

By default, a column can hold NULL values. The `NOT NULL` constraint enforces a column to NOT accept NULL values. This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.

The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values when the "Persons" table is created:

Example:

```
CREATE TABLE Persons (ID int NOT NULL,LastName varchar(255)NOT NULL,
FirstName varchar(255) NOT NULL,Age int);
```

- **SQL NOT NULL on ALTER TABLE:-**

To create a `NOT NULL` constraint on the "Age" column when the "Persons" table is already created, use the following SQL:

```
ALTER TABLE Persons MODIFY COLUMN Age int NOT NULL;
```

2. SQL UNIQUE Constraint:-

The `UNIQUE` constraint ensures that all values in a column are different.
Both the `UNIQUE` and `PRIMARY KEY` constraints provide a guarantee for uniqueness for a column or set of columns.
A `PRIMARY KEY` constraint automatically has a `UNIQUE` constraint.
However, you can have many `UNIQUE` constraints per table, but only one `PRIMARY KEY` constraint per table.
he following SQL creates a `UNIQUE` constraint on the "ID" column when the "Persons" table is created:

```
CREATE TABLE Persons (ID int NOT NULL UNIQUE,LastName varchar(255) NOT
NULL,FirstName varchar(255),Age int);
```

- **SQL UNIQUE Constraint on ALTER TABLE:-**

To create a `UNIQUE` constraint on the "ID" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons ADD UNIQUE (ID);
```

3. SQL PRIMARY KEY Constraint:-

The `PRIMARY KEY` constraint is used to uniquely identify each record in a table.
Primary keys must contain unique values, and cannot contain NULL values.
Each table can have only ONE primary key. The primary key can be a single column or a combination of columns.
The following SQL creates a `PRIMARY KEY` on the "ID" column when the "Persons" table is created:

```
CREATE TABLE Persons (ID int NOT NULL,LastName varchar(255) NOT NULL,
FirstName varchar(255), Age int, PRIMARY KEY (ID) );
```

To define a `PRIMARY KEY` constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons(ID int NOT NULL,LastName varchar(255) NOT NULL,FirstName
varchar(255),Age int,CONSTRAINT PK_Person PRIMARY KEY (ID,LastName));
```

Create a primary key for the Persons table, name this primary key as PK_Person, and this primary key will be formed using both ID and LastName.

4. SQL FOREIGN KEY Constraint:-

The `FOREIGN KEY` constraint is used to prevent actions that would destroy links between tables.
A `FOREIGN KEY` is a field (or collection of fields) in one table, that refers to the `PRIMARY KEY` in another table.
The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.
Look at the following two tables:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

FOREIGN KEY

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table.
The "PersonID" column in the "Persons" table is the `PRIMARY KEY` in the "Persons" table.
The "PersonID" column in the "Orders" table is a `FOREIGN KEY` in the "Orders" table.
The `FOREIGN KEY` constraint prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the parent table.

- **SQL FOREIGN KEY on CREATE TABLE**

  The following SQL creates a `FOREIGN KEY` on the "PersonID" column when the "Orders" table is created:

```
CREATE TABLE Orders (OrderID int NOT NULL,OrderNumber int NOT NULL,PersonID int,PRIMARY KEY (OrderID),FOREIGN KEY (PersonID) REFERENCES Persons(PersonID));
```

- **SQL FOREIGN KEY on ALTER TABLE**

  To create a `FOREIGN KEY` constraint on the "PersonID" column when the "Orders" table is already created, use the following SQL:

```
ALTER TABLE Orders ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

5. SQL CHECK Constraint:-

The `CHECK` constraint is used to limit the value range that can be placed in a column.

If you define a `CHECK` constraint on a column it will allow only certain values for this column.

If you define a `CHECK` constraint on a table it can limit the values in certain columns based on values in other columns in the row.

The following SQL creates a `CHECK` constraint on the "Age" column when the "Persons" table is created. The `CHECK` constraint ensures that the age of a person must be 18, or older:

```
CREATE TABLE Persons (ID int NOT NULL,LastName varchar(255) NOT NULL,FirstName varchar(255),Age int,CHECK (Age>=18));
```

- **SQL CHECK on ALTER TABLE**

To create a `CHECK` constraint on the "Age" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons ADD CHECK (Age>=18);
```

6. SQL DEFAULT Constraint:-

The `DEFAULT` constraint is used to set a default value for a column. The default value will be added to all new records, if no other value is specified.

The following SQL sets a `DEFAULT` value for the "City" column when the "Persons" table is created:

```
CREATE TABLE Persons (ID int NOT NULL,LastName varchar(255) NOT
NULL,FirstName varchar(255),Age int,City varchar(255)DEFAULT 'Sandnes');
```

- **SQL DEFAULT on ALTER TABLE**

To create a `DEFAULT` constraint on the "City" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons ALTER City SET DEFAULT 'Sandnes';
```

7. SQL AUTO INCREMENT Field:-

Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table.Often this is the primary key field that we would like to be created automatically every time a new record is inserted.

```
CREATE TABLE Persons (Personid int NOT NULL AUTO_INCREMENT,LastName
varchar(255) NOT NULL,FirstName varchar(255),Age int,PRIMARY KEY (Personid));
```

MySQL uses the `AUTO_INCREMENT` keyword to perform an auto-increment feature. By default, the starting value for `AUTO_INCREMENT` is 1, and it will increment by 1 for each new record. To let the `AUTO_INCREMENT` sequence start with another value, use the following SQL statement:

```
ALTER TABLE Persons AUTO_INCREMENT=100;
```

To insert a new record into the "Persons" table, we will NOT have to specify a value for the "Personid" column (a unique value will be added automatically):

```
INSERT INTO Persons (FirstName,LastName) VALUES ('Lars','Monsen');
```

8. SQL Dates:-

MySQL comes with the following data types for storing a date or a date/time value in the database:

- `DATE` - format YYYY-MM-DD
- `DATETIME` - format: YYYY-MM-DD HH:MI:SS
- `TIMESTAMP` - format: YYYY-MM-DD HH:MI:SS
- `YEAR` - format YYYY or YY

SQL Working with Dates:- Look at the following table:

Orders Table:-

OrderId	ProductName	OrderDate
1	Geitost	2008-11-11
2	Camembert Pierrot	2008-11-09
3	Mozzarella di Giovanni	2008-11-11
4	Mascarpone Fabioli	2008-10-29

Now we want to select the records with an OrderDate of "2008-11-11" from the table above.We use the following `SELECT` statement:

```
SELECT * FROM Orders WHERE OrderDate='2008-11-11'
```

SQL Syntax:-

SQL Statements:-

Most of the actions you need to perform on a database are done with SQL statements. SQL statements consist of keywords that are easy to understand. The following SQL statement returns all records from a table named "Customers":

```
SELECT * FROM Customers;
```

Keep in Mind That...

- SQL keywords are NOT case sensitive: `select` is the same as `SELECT`

Some of The Most Important SQL Commands:-

- `SELECT` - extracts data from a database
- `UPDATE` - updates data in a database
- `DELETE` - deletes data from a database
- `INSERT INTO` - inserts new data into a database
- `CREATE DATABASE` - creates a new database
- `ALTER DATABASE` - modifies a database
- `CREATE TABLE` - creates a new table
- `ALTER TABLE` - modifies a table
- `DROP TABLE` - deletes a table
- `CREATE INDEX` - creates an index (search key)
- `DROP INDEX` - deletes an index

The SQL SELECT Statement:-

The `SELECT` statement is used to select data from a database.

Syntax:

```
SELECT Column1,Column2,... FROM table_name;
```

Example:

```
SELECT personId,name FROM Persons;
```

Here, column1, column2, ... are the *field names* of the table you want to select data from.
The table_name represents the name of the *table* you want to select data from.

Select ALL columns:-

If you want to return all columns, without specifying every column name, you can use the `SELECT *` syntax:

Example

Return all the columns from the Customers table:

```
SELECT * FROM Persons;
```

The SQL WHERE Clause:-

The `WHERE` clause is used to filter records. It is used to extract only those records that fulfill a specified condition.

Syntax:

```
SELECT column1, column2, … FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM Customers WHERE Country='Mexico';
```

Note: The `WHERE` clause is not only used in `SELECT` statements, it is also used in `UPDATE`, `DELETE`, etc.!

Select ALL columns:-

If you want to return all columns, without specifying every column name, you can use the `SELECT *` syntax:

Example

Return all the columns from the Customers table:

```
SELECT * FROM Persons;
```

SQL Order BY:-

The `ORDER BY` keyword is used to sort the result-set in ascending or descending order.

Example

Sort the products by price:

```
SELECT * FROM Products ORDER BY Price ASC/DESC;
```

The SQL AND Operator:-

The `WHERE` clause can contain one or many `AND` operators.
The `AND` operator is used to filter records based on more than one condition, like if you want to return all customers from Spain that starts with the letter 'G':

```
SELECT * FROM Customers WHERE Country = 'Spain' AND CustomerName LIKE 'G%';
```

AND vs OR:-

The `AND` operator displays a record if *all* the conditions are TRUE. The `OR` operator displays a record if *any* of the conditions are TRUE.

The SQL OR Operator:-

The `WHERE` clause can contain one or more `OR` operators.
The `OR` operator is used to filter records based on more than one condition, like if you want to return all customers from Germany but also those from Spain:

```
SELECT * FROM Customers WHERE Country = 'Germany' OR Country = 'Spain';
```

The NOT Operator:-

The `NOT` operator is used in combination with other operators to give the opposite result, also called the negative result.
In the select statement below we want to return all customers that are NOT from Spain:
Select only the customers that are NOT from Spain:

```
SELECT * FROM Customers WHERE NOT Country = 'Spain';
```

The SQL INSERT INTO Statement:-

The `INSERT INTO` statement is used to insert new records in a table.

INSERT INTO Syntax:-

It is possible to write the `INSERT INTO` statement in two ways:

1. Specify both the column names and the values to be inserted:

Insert Data Only in Specified Columns:-

It is also possible to only insert data in specific columns.

Syntax:

```
INSERT INTO table_name (column1, column2, column3, ...) VALUES
(value1, value2, value3, ...);
```

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table. Here, the `INSERT INTO` syntax would be as follows:

Example:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City,
PostalCode, Country) VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21',
'Stavanger', '4006', 'Norway');
```

3. It is also possible to insert multiple rows in one statement. To insert multiple rows of data, we use the same `INSERT INTO` statement, but with multiple values:

Example:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode,
Country) VALUES
('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway'),
('Greasy Burger', 'Per Olsen', 'Gateveien 15', 'Sandnes', '4306', 'Norway'),
('Tasty Tee', 'Finn Egan', 'Streetroad 19B', 'Liverpool', 'L1 0AA', 'UK');
```

What is a NULL Value?

A field with a NULL value is a field with no value. If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.

Note: A NULL value is different from a zero value or a field that contains spaces. A field with a NULL value is one that has been left blank during record creation!

How to Test for NULL Values?

It is not possible to test for NULL values with comparison operators, such as =, <, or <>. We will have to use the `IS NULL` and `IS NOT NULL` operators instead.

Syntax:

```
SELECT column_names FROM table_name WHERE column_name IS NULL/IS NOT NULL;
```

The `IS NULL` operator is used to test for empty values (NULL values). The following SQL lists all customers with a NULL value in the "Address" field:

Example:

```
SELECT CustomerName, ContactName, Address FROM Customers WHERE Address IS NULL;
```

Tip: Always use IS NULL to look for NULL values.

The IS NOT NULL Operator:-

The `IS NOT NULL` operator is used to test for non-empty values (NOT NULL values). The following SQL lists all customers with a value in the "Address" field:

Example:

```
SELECT CustomerName, ContactName, Address FROM Customers WHERE Address IS NOT NULL;
```

The SQL UPDATE Statement:-

The `UPDATE` statement is used to modify the existing records in a table.

Syntax:

```
UPDATE table_name SET column1 = value1, column2 = value2, … WHERE condition;
```

Note: Be careful when updating records in a table! Notice the `WHERE` clause in the `UPDATE` statement. The `WHERE` clause specifies which record(s) that should be updated. If you omit the `WHERE` clause, all records in the table will be updated!

Example:

```
UPDATE Customers SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

UPDATE Multiple Records:-

It is the `WHERE` clause that determines how many records will be updated.

The following SQL statement will update the ContactName to "Juan" for all records where country is "Mexico":

Update Warning!:-

Be careful when updating records. If you omit the `WHERE` clause, ALL records will be updated!

Example:

```
UPDATE Customers SET ContactName='Juan';
```

The SQL DELETE Statement:-

The `DELETE` statement is used to delete existing records in a table.

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Note: Be careful when deleting records in a table! Notice the `WHERE` clause in the `DELETE` statement. The `WHERE` clause specifies which record(s) should be deleted. If you omit the `WHERE` clause, all records in the table will be deleted!

Example:

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

Delete All Records:-

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

The following SQL statement deletes all rows in the "Customers" table, without deleting the table:

Syntax:

```
DELETE FROM table_name;
```

Example:

```
DELETE FROM Customers;
```

The SQL Limit Statement:-

The **LIMIT** statement in SQL is used to **restrict the number of rows** returned by a query.

It is mainly used when:

- You want to see **only a few records**
- You are working with **large tables**
- You want **pagination** (show records page-wise)

Syntax:

```
SELECT column1, column2 FROM table_name LIMIT number;
```

Example:

```
SELECT * FROM Customers LIMIT 5;
```

Show top 2 students by marks:-

Example:

```
SELECT * FROM students ORDER BY marks DESC LIMIT 2;
```

OFFSET is used in SQL to skip a specified number of rows before starting to return rows from the result set.

OFFSET tells SQL how many rows to ignore first.

It is mostly used **with LIMIT** for:

- Pagination (page-wise data)
- Skipping records
- Showing data in parts

Skip first 2 rows and show next 2 rows:-

Example:

```
SELECT * FROM students LIMIT 2 OFFSET 2;
```

SQL Aggregate Functions:-

An aggregate function is a function that performs a calculation on a set of values, and returns a single value.
Aggregate functions are often used with the `GROUP BY` clause of the `SELECT` statement. The `GROUP BY` clause splits the result-set into groups of values and the aggregate function can be used to return a single value for each group.
The most commonly used SQL aggregate functions are:

- `MIN()` - returns the smallest value within the selected column
- `MAX()` - returns the largest value within the selected column
- `COUNT()` - returns the number of rows in a set
- `SUM()` - returns the total sum of a numerical column
- `AVG()` - returns the average value of a numerical column

Aggregate functions ignore null values (except for `COUNT(*)`).
We will go through the aggregate functions above in the next chapters.

The SQL **MIN()** and **MAX()** Functions:-

MIN() Function:-

The **MIN()** function is an **aggregate function** in SQL.
It returns the **smallest (minimum) value** from a selected column.
It works with:

- Numbers
- Dates
- Strings (alphabetically smallest)

Syntax:

```
SELECT MIN(column_name) FROM table_name;
```

Example:

```
SELECT MIN(marks) AS Minimum_Marks FROM students;
```

Why do we use MIN() with GROUP BY?

When we want:

- The **minimum value for each group**, not for the whole table

Without GROUP BY you get **one minimum value** for the entire table. With GROUP BY you get **separate minimum values for each group**.

Example:

```
SELECT category, MIN(price) AS Minimum_Price FROM Products GROUP BY category;
```

Why use MAX() with GROUP BY?:-

When we want:

- The **highest value for each group**
- Not just one maximum for the whole table

Example:

- Highest price in **Electronics**
- Highest price in **Clothing**
- Highest price in **Grocery**

Example:

```
SELECT category, MAX(price) AS Maximum_Price FROM Products GROUP BY category;
```

The SQL COUNT() Function:-

The `COUNT()` function returns the number of rows that matches a specified criterion.

Syntax:

```
SELECT COUNT(column_name) FROM table_name WHERE condition;
```

Example:

```
SELECT COUNT(*) FROM Products;
```

The SQL SUM() Function:-

The `SUM()` function returns the total sum of a numeric column.

Syntax:

```
SELECT SUM(column_name)FROM table_name WHERE condition;
```

Example:

```
SELECT SUM(Quantity) FROM OrderDetails;
```

The SQL AVG() Function:-

The `AVG()` function returns the average value of a numeric column.

Syntax:

```
SELECT AVG(column_name) FROM table_name WHERE condition;
```

Example:

```
SELECT AVG(Price) FROM Products;
```

The SQL LIKE Operator:-

The `LIKE` operator is used in a `WHERE` clause to search for a specified pattern in a column. There are two wildcards often used in conjunction with the `LIKE` operator:

- The percent sign `%` represents zero, one, or multiple characters
- The underscore sign `_` represents one, single character

Syntax:

```
SELECT column1, column2,… FROM table_name WHERE column LIKE pattern;
```

Select all customers that starts with the letter "a":

Example:

```
SELECT * FROM Customers WHERE CustomerName LIKE 'a%';
```

Return all customers from a city that *contains* the letter 'D':

Example:

```
SELECT * FROM Customers WHERE city LIKE '%D%';
```

The _ Wildcard:-

The _ wildcard represents a single character. It can be any character or number, but each _ represents one, and only one, character.

Return all customers from a city that starts with 'D' followed by one wildcard character, then 'lh' and then two wildcard characters:

Example:

```
SELECT * FROM Customers WHERE city LIKE 'D_lh__';
```

The SQL IN Operator:-

The `IN` operator allows you to specify multiple values in a `WHERE` clause. The `IN` operator is a shorthand for multiple `OR` conditions.

Syntax:

```
SELECT column_name(s) FROM table_name WHERE column_name IN (value1, value2, ...);
```

Example:

```
SELECT * FROM Customers WHERE Country IN ('Germany', 'France', 'UK');
```

NOT IN:-

By using the `NOT` keyword in front of the `IN` operator, you return all records that are NOT any of the values in the list.

Example:

```
SELECT * FROM Customers WHERE Country NOT IN ('Germany', 'France', 'UK');
```

The SQL BETWEEN Operator:-

The `BETWEEN` operator selects values within a given range. The values can be numbers, text, or dates. The `BETWEEN` operator is inclusive: begin and end values are included.

Syntax:

```
SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2;
```

Example:

```
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;
```

SQL Aliases:-

SQL aliases are used to give a table, or a column in a table, a temporary name. Aliases are often used to make column names more readable. An alias only exists for the duration of that query. An alias is created with the `AS` keyword.

What is an Alias in SQL?:-

An **alias** in SQL is a **temporary name** given to:

- a **column**
- or a **table**

It is used to:

- make output **more readable**
- shorten long column or table names
- improve query clarity

Example:

```
SELECT CustomerID AS ID FROM Customers;
```

Types of Alias in SQL:-

1. Column Alias
2. Table Alias

Example:

```
SELECT column_name AS alias_name FROM table_name AS alias_name;
```

Having statement in SQL:-

HAVING is a clause in SQL used to **filter grouped records** after applying **aggregate functions** like `COUNT()`, `SUM()`, `MIN()`, `MAX()`, and `AVG()`.

Why HAVING is Needed?

- `WHERE` **cannot** be used with aggregate functions
- `HAVING` is used **with GROUP BY**
- It filters the **result of aggregate functions**

Syntax:

```
SELECT column_name, aggregate_function(column_name) FROM table_name
GROUP BY column_name HAVING condition;
```

Table:- Products

id	product	category	price
1	Laptop	Electronics	50000
2	Mobile	Electronics	20000
3	TV	Electronics	45000
4	Shirt	Clothing	1200
5	Jeans	Clothing	1800
6	Shoes	Clothing	1500
7	Rice	Grocery	900
8	Sugar	Grocery	600

Example 1: Find the cheapest product price in each category

Query:

```
SELECT category, MIN(price) AS Min_Price FROM Products GROUP BY
category;
```

Example 2: Find the highest price in each category (price > 1000 only)

Query:

```
SELECT category, MAX(price) AS Max_Price FROM Products WHERE price >
1000 GROUP BY category;
```

Example 3: Show categories whose minimum price is below 1500

Query:

```
SELECT category, MIN(price) AS Min_Price FROM Products GROUP BY
category HAVING MIN(price) < 1500;
```

Example 4: Show categories sorted by highest price (descending)

Query:

```
SELECT category, MAX(price) AS Max_Price FROM Products GROUP BY
category ORDER BY Max_Price DESC;
```

Example 5: Show top 2 categories with highest-priced products

Query:

```
SELECT category, MAX(price) AS Max_Price FROM Products GROUP BY
category ORDER BY Max_Price DESC LIMIT 2;
```

SQL Joins:-

In SQL, **JOIN** is used to combine data from **two or more tables** based on a **related column** between them. Generally, tables are connected using a **primary key** in one table and a **foreign key** in another table. JOIN helps us retrieve meaningful information that is spread across multiple tables instead of storing all data in a single table. This makes the database **organized, efficient, and easy to manage**.

There are different types of JOINs in SQL. The most commonly used JOIN is **INNER JOIN**, which returns only those records that have matching values in both tables. **LEFT JOIN** returns all records from the left table and matching records from the right table; if there is no match, NULL values are shown. **RIGHT JOIN** works opposite to LEFT JOIN, returning all records from the right table. **FULL JOIN** returns all records from both tables, whether there is a match or not.

department

dept_id	dept_name
1	HR
2	IT
3	Finance

employee

emp_id	emp_name	dept_id
101	Amit	1
102	Neha	2
103	Kavita	null

To get employee names along with their department names, we use JOIN:

```
SELECT * FROM employee INNER JOIN department ON employee.dept_id = department.dept_id;
```

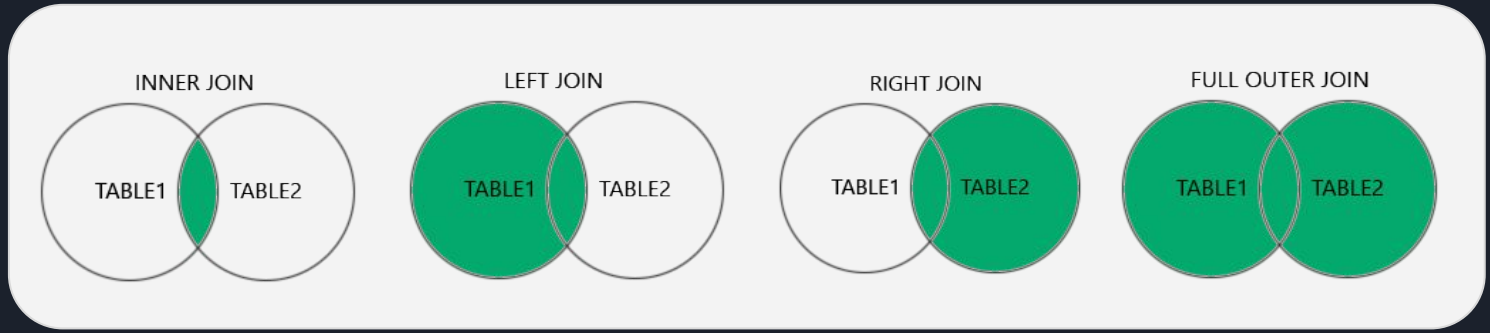
This query combines both tables and shows only matching department records, making JOIN a powerful tool in SQL.

emp_id	emp_name	dept_id	dept_id	dept_name
101	Amit	1	1	HR
102	Neha	2	2	IT

Different Types of SQL JOINs:-

Here are the different types of the JOINs in SQL:

- `(INNER) JOIN` Returns records that have matching values in both tables.
- `LEFT (OUTER) JOIN` Returns all records from the left table, and the matched records from the right table.
- `RIGHT (OUTER) JOIN` Returns all records from the right table, and the matched records from the left table.
- `FULL (OUTER) JOIN` Returns all records when there is a match in either left or right table.



INNER JOIN:

INNER JOIN is used to fetch only those records that have **matching values in both tables**. If a row in one table does not have a related row in the other table, it will **not appear** in the result. INNER JOIN is the **most commonly used JOIN** in SQL because it returns only meaningful and related data. It helps reduce unnecessary records and improves query accuracy. INNER JOIN works on the condition specified in the ON clause, usually using primary and foreign keys.

Syntax:

```
SELECT columns FROM table1 INNER JOIN table2 ON table1.common_column
= table2.common_column;
```

Example:

```
SELECT * FROM employee e INNER JOIN department d ON e.dept_id =
d.dept_id;
```

Output:

emp_id	emp_name	dept_id	dept_id	dept_name
101	Amit	1	1	HR
102	Neha	2	2	IT

When we join two tables, SQL creates combinations of rows.
The **ON condition filters those combinations** and joins only the rows where the condition is true.
Most of the time, this condition connects:

- **Primary Key** of one table
- with **Foreign Key** of another table

LEFT JOIN (LEFT OUTER JOIN):-

LEFT JOIN returns **all records from the left table** and only the **matching records from the right table**. If there is no match in the right table, SQL returns **NULL values** for the right table columns. LEFT JOIN is useful when we want **complete data from the main table**, even if related data is missing. It is commonly used in reports where missing relationships must still be shown.

Syntax:

```
SELECT columns FROM table1 LEFT JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT * FROM employee e LEFT JOIN department d ON e.dept_id = d.dept_id;
```

Output:

emp_id	emp_name	dept_id	dept_id	dept_name
101	Amit	1	1	HR
102	Neha	2	2	IT
103	Kavita	null	null	null

RIGHT JOIN (RIGHT OUTER JOIN):-

RIGHT JOIN returns **all records from the right table** and only the **matching records from the left table**. If there is no matching row in the left table, the result will contain **NULL values** for left table columns. RIGHT JOIN is less commonly used because the same result can usually be achieved using LEFT JOIN by swapping table positions. It is helpful when the right table is more important.

Syntax:

```
SELECT columns FROM table1 RIGHT JOIN table2 ON table1.common_column
= table2.common_column;
```

Example:

```
SELECT * FROM employee e RIGHT JOIN department d ON e.dept_id =
d.dept_id;
```

Output:

emp_id	emp_name	dept_id	dept_id	dept_name
101	Amit	1	1	HR
102	Neha	2	2	IT
null	null	null	3	Finance

FULL JOIN (LEFT OUTER JOIN):-

FULL JOIN returns **all records from both tables**, whether there is a match or not. If a matching row exists, data is combined; otherwise, **NULL values** are shown for missing columns. FULL JOIN is useful when we want a **complete comparison** of two tables. It helps identify unmatched records in both tables. Note that some databases (like MySQL) do not directly support FULL JOIN.

Note:

MySQL does NOT support FULL JOIN (FULL OUTER JOIN).
When MySQL sees FULL JOIN, it **doesn't recognize it as a valid join type**, so it treats FULL incorrectly and throws an error (often **1054: Unknown column** or similar).
So to get all data like FULL JOIN we can use UNION between LEFT JOIN and RIGHT JOIN Query.

Example:

```
SELECT * FROM employee e RIGHT JOIN department d ON e.dept_id =
d.dept_id;
UNION
SELECT * FROM employee e RIGHT JOIN department d ON e.dept_id =
d.dept_id;
```

Output:

emp_id	emp_name	dept_id	dept_id	dept_name
101	Amit	1	1	HR
102	Neha	2	2	IT
103	Kavita	null	null	null
null	null	null	3	Finance

The SQL UNION Operator:-

The `UNION` operator is used to combine the result-set of two or more `SELECT` statements. The `UNION` operator automatically removes duplicate rows from the result set.

Requirements for `UNION`:

- Every `SELECT` statement within `UNION` must have the same number of columns.
- The columns must also have similar data types.
- The columns in every `SELECT` statement must also be in the same order.

Syntax:

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

Example:

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers ORDER BY City;
```

Note: The column names in the result-set are usually equal to the column names in the first `SELECT` statement.

Note: If some customers or suppliers have the same city, each city will only be listed once, because `UNION` selects only distinct values. Use `UNION ALL` to also select duplicate values!

The SQL EXISTS Operator:-

The `EXISTS` operator is used to test for the existence of any record in a subquery. The `EXISTS` operator returns TRUE if the subquery returns one or more records.

Syntax:

```
SELECT column_name(s) FROM table_name
WHERE EXISTS (SELECT column_name FROM table_name WHERE condition);
```

Example:

```
SELECT SupplierName
FROM Suppliers
WHERE EXISTS (SELECT ProductName FROM Products WHERE Products.SupplierID
= Suppliers.supplierID AND Price < 20);
```

Note:- It displays the names of suppliers who have **at least one product** with a **price less than 20**.

EXISTS = “Does at least one record exist?”

- If the subquery returns **even one row** → `EXISTS` is **TRUE**
- If the subquery returns **no rows** → `EXISTS` is **FALSE**

`EXISTS` does **not** return data

It only checks whether a condition is satisfied or not

Table 1: `department`

```
CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

INSERT INTO department VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Marketing');
```

Table 2: `employee`

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary INT,
    hire_date DATE,
    city VARCHAR(50),
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

```
INSERT INTO employee VALUES
(101, 'Rohit', 3, 60000, '2020-04-12', 'Delhi'),
(102, 'Sneha', 1, 45000, '2019-09-15', 'Mumbai'),
(103, 'Amit', 2, 55000, '2021-06-20', 'Delhi'),
(104, 'Neha', 3, 72000, '2018-11-30', 'Pune'),
(105, 'Vikas', 4, 38000, '2022-03-05', 'Delhi'),
(106, 'Priya', 1, 48000, '2021-07-11', 'Chandigarh'),
(107, 'Ankit', 3, 53000, '2023-01-19', 'Noida'),
(108, 'Kiran', 4, 41000, '2020-05-22', 'Pune'),
(109, 'Manish', 2, 64000, '2022-12-10', 'Delhi'),
(110, 'Deepika', 3, 58000, '2019-02-25', 'Chandigarh');
```

Table 3: project

```
CREATE TABLE project (
    proj_id INT PRIMARY KEY,
    proj_name VARCHAR(50),
    dept_id INT,
    budget INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

```
INSERT INTO project VALUES
(201, 'Payroll System', 2, 800000),
(202, 'Website Revamp', 3, 600000),
(203, 'Recruitment Portal', 1, 400000),
(204, 'Ad Campaign', 4, 300000),
(205, 'Cloud Migration', 3, 900000);
```

Table 4: works_on

```
CREATE TABLE works_on (
    emp_id INT,
    proj_id INT,
    hours_worked INT,
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id),
    FOREIGN KEY (proj_id) REFERENCES project(proj_id)
);

INSERT INTO works_on VALUES
(101, 202, 120),
(101, 205, 100),
(102, 203, 150),
(103, 201, 80),
(104, 205, 200),
(105, 204, 90),
(106, 203, 60),
(107, 202, 70),
(108, 204, 110),
(109, 201, 130),
(110, 202, 85);
```

Questions:-

1. Display all records from the **employee** table.
2. Show employee names and salaries only.
3. List all departments from the **department** table.
4. Display all employees who belong to the **IT department**.
5. Find employees whose salary is greater than **50,000**.
6. Display employees who live in **Delhi**.
7. Show employee names and hire dates.
8. List all projects with their budget.
9. Display projects having a budget greater than **5,00,000**.
10. Show all employees hired after **2020-01-01**.
11. Display employee name along with their **department name**.
12. List employee name and project name they are working on.
13. Show department name and project name for all projects.
14. Display employee name and hours worked on projects.
15. List all employees working on the **Website Revamp** project.
16. Count total number of employees.
17. Find the **highest salary** among employees.
18. Find the **average salary** of employees.
19. Count how many employees work in each department.
20. Show total number of projects in the company.
21. Display department name and **average salary** of employees in each department.
22. List employees who are working on **more than one project**.
23. Find total hours worked by each employee.
24. Display project name and **total hours worked** on each project.
25. List employees who are working on projects with a **budget greater than 6,00,000**.
26. Find employees whose salary is **greater than the average salary**.
27. Display department names that have **more than 2 employees**.
28. List employees who are **not assigned to any project** (if any).
29. Find the department which has the **maximum number of projects**.
30. Display employee names who work in the **same department as Rohit**.

Advance Level:-

1. Display all employees along with their department names.
2. List employees who work in **IT department** and have salary > 55000.
3. Show all projects handled by the **Finance** department.
4. Find total salary paid in each department.
5. Show employee names and number of projects each employee is working on.
6. Find the average salary of employees in each department.
7. Display department names having **average salary > 50000**.
8. Show employees who joined after year 2020.
9. List all employees from **Delhi** working on any project.
10. Display employees who are not assigned to any project.
11. Find employees who earn more than the **average salary** of the company.
12. Show the **highest-paid employee** in each department.
13. Display employees working on projects with **budget > 700000**.
14. Show project names where **at least 2 employees** are working.
15. Display employee(s) who have worked the **maximum hours** overall.
16. Show all employees whose salary is greater than **Rohit's salary**.
17. Find departments that have **no projects assigned**.
18. Show the **second-highest salary** in each department.
19. Display employees who work on the **same project** as 'Amit'.
20. List employees who work on **both project 202 and 205**.
21. Find total hours worked by each employee on all projects.
22. Display project names and total hours spent by employees.
23. Show departments with **more than 2 employees**.
24. Find employees who have worked **more than 100 hours** in total.
25. Show department name and total project budget handled by it.
26. Find employees who work on **projects of their own department** only.
27. List employees whose **department has a budget > 600000** in total.
28. Display department name, total salary, and total hours worked (combine data from all 3 tables).
29. Find the **top 3 highest-paid employees** along with department names.
30. Display employees grouped by city, showing **average salary per city**.

Thankyou!